In [1]:
```python
#1
import numpy as np

# Create the two 1-D NumPy arrays
array1 = np.array([1, 2, 3, 0])
array2 = np.array([4, 1, 0, 6])

# Addition
addition_result = array1 + array2

# Subtraction
subtraction_result = array1 - array2

# Multiplication
multiplication_result = array1 * array2

# Division
division_result = np.divide(array1, array2, where=array2 != 0)

# Print the results
print("Addition:", addition_result)
print("Subtraction:", subtraction_result)
print("Multiplication:", multiplication_result)
print("Division:", division_result)
```

```
Addition: [5 3 3 6]
Subtraction: [-3  1  3 -6]
Multiplication: [4 2 0 0]
Division: [0.25 2.   0.   0.  ]
```

In [11]:
```python
#2.
import numpy as np

# Create the 1-D numpy array with 5 random integers between 1 and 100
array1 = np.random.randint(1, 101, size=5)

# Create the 2-D numpy array with shape (4, 5) and random integers between 1 and 100
array2 = np.random.randint(1, 101, size=(4, 5))
```

```
# Perform element-wise addition (broadcasting will occur)
addition_result = array2 + array1

# Perform element-wise subtraction (broadcasting will occur)
subtraction_result = array2 - array1

# Print the results
print("1-D Array (array1):")
print(array1)
print("\n2-D Array (array2):")
print(array2)

print("\nAddition result (array2 + array1):")
print(addition_result)

print("\nSubtraction result (array2 - array1):")
print(subtraction_result)
```

```
1-D Array (array1):
[76 83 19 41 89]

2-D Array (array2):
[[ 2 91 36 12 77]
 [38 45 41 52 34]
 [24 70  5 97 12]
 [16 90 62 73 36]]

Addition result (array2 + array1):
[[ 78 174  55  53 166]
 [114 128  60  93 123]
 [100 153  24 138 101]
 [ 92 173  81 114 125]]

Subtraction result (array2 - array1):
[[-74   8  17 -29 -12]
 [-38 -38  22  11 -55]
 [-52 -13 -14  56 -77]
 [-60   7  43  32 -53]]
```

In [5]:
```
#3.
import numpy as np
```

```
# Create a 3-D array of shape (3, 3, 3) with random integers between 1 and 100
array = np.random.randint(1, 101, size=(3, 3, 3))

# Print the original 3-D array
print("Original 3-D Array:")
print(array)

# Extract the third column of the second 2-D array
third_column_second_2d = array[1, :, 2]

# Extract the first and third row of the first 2-D array
first_and_third_row_first_2d = array[0, [0, 2], :]

# Extract the intersection of the first row and third column in the third 2-D array
intersection_first_row_third_column_third_2d = array[2, 0, 2]

# Print the results
print("\nThird column of the second 2-D array:")
print(third_column_second_2d)

print("\nFirst and third row of the first 2-D array:")
print(first_and_third_row_first_2d)

print("\nIntersection of the first row and third column in the third 2-D array:")
print(intersection_first_row_third_column_third_2d)
```

```
Original 3-D Array:
[[[ 97  11  94]
  [  3  37  91]
  [  5  89  78]]

 [[ 23  35  24]
  [ 45  59  90]
  [ 53  84  25]]

 [[ 32  22 100]
  [ 47  56  98]
  [ 95   1  69]]]

Third column of the second 2-D array:
[24 90 25]

First and third row of the first 2-D array:
[[97 11 94]
 [ 5 89 78]]

Intersection of the first row and third column in the third 2-D array:
100
```

In [7]:

```python
#4
import numpy as np

# Step 1: Create a 2-D numpy array with random integers between 1 and 100 with shape (5, 4)
array_2d = np.random.randint(1, 101, size=(5, 4))

# Print the original array
print("Original 2-D Array:")
print(array_2d)

# Step 2: Print only the even numbers from the array
even_numbers = array_2d[array_2d % 2 == 0]
print("\nEven numbers from the array:")
print(even_numbers)

# Step 3: Print only the odd numbers from the array
odd_numbers = array_2d[array_2d % 2 != 0]
print("\nOdd numbers from the array:")
```

```
print(odd_numbers)

# Step 4: Print row-wise sum of all elements
row_sum = array_2d.sum(axis=1)
print("\nRow-wise sum of all elements:")
print(row_sum)

# Step 5: Print column-wise sum of all elements
column_sum = array_2d.sum(axis=0)
print("\nColumn-wise sum of all elements:")
print(column_sum)

# Step 6: Convert this 2-D array into a 3-D array of shape (2, 2, 5)
array_3d = array_2d.reshape(2, 2, 5)
print("\nConverted 3-D Array (shape 2x2x5):")
print(array_3d)
```

```
Original 2-D Array:
[[47 49  4 74]
 [61 42  2 77]
 [99 15 96 39]
 [45 97 22 92]
 [ 8 32 19 83]]

Even numbers from the array:
[ 4 74 42  2 96 22 92  8 32]

Odd numbers from the array:
[47 49 61 77 99 15 39 45 97 19 83]

Row-wise sum of all elements:
[174 182 249 256 142]

Column-wise sum of all elements:
[260 235 143 365]

Converted 3-D Array (shape 2x2x5):
[[[47 49  4 74 61]
  [42  2 77 99 15]]

 [[96 39 45 97 22]
  [92  8 32 19 83]]]
```

In [9]:

```python
#5.
import numpy as np

# Step 1: Create a 2-D numpy array with random integers between 1 and 100 with shape (5, 4)
array_2d = np.random.randint(1, 101, size=(5, 4))

# Print the original 2-D array
print("Original 2-D Array:")
print(array_2d)

# Step 2: Swap the first row with the third row
array_2d[[0, 2]] = array_2d[[2, 0]]
print("\nArray after swapping first and third rows:")
print(array_2d)
```

```
# Step 3: Swap the second column with the fourth column
array_2d[:, [1, 3]] = array_2d[:, [3, 1]]
print("\nArray after swapping second and fourth columns:")
print(array_2d)

# Step 4: Replace values less than 50 with zero
array_2d[array_2d < 50] = 0
print("\nArray after replacing values less than 50 with zero:")
print(array_2d)

# Step 5: Convert this 2-D array into a 1-D array
array_1d = array_2d.flatten()
print("\nConverted 1-D Array:")
print(array_1d)
```

```
Original 2-D Array:
[[ 3 70 49 56]
 [63 44 22 71]
 [59 38 15 87]
 [24 97 77 37]
 [26 83 71 77]]

Array after swapping first and third rows:
[[59 38 15 87]
 [63 44 22 71]
 [ 3 70 49 56]
 [24 97 77 37]
 [26 83 71 77]]

Array after swapping second and fourth columns:
[[59 87 15 38]
 [63 71 22 44]
 [ 3 56 49 70]
 [24 37 77 97]
 [26 77 71 83]]

Array after replacing values less than 50 with zero:
[[59 87  0  0]
 [63 71  0  0]
 [ 0 56  0 70]
 [ 0  0 77 97]
 [ 0 77 71 83]]

Converted 1-D Array:
[59 87  0  0 63 71  0  0  0 56  0 70  0  0 77 97  0 77 71 83]
```

In [ ]:
```
#6
Stacking refers to the process of combining multiple arrays along a new axis. In NumPy, stacking can be done using
Vertical Stacking (np.vstack()): This stacks arrays on top of each other (along rows, axis=0).
Horizontal Stacking (np.hstack()): This stacks arrays side by side (along columns, axis=1).
Depth Stacking (np.dstack()): This stacks arrays along a new depth axis (axis=2).
Examples of Stacking:
import numpy as np
Examples:
# Two 2-D arrays with the same number of columns
arr1 = np.array([[1, 2], [3, 4]])
```

```
arr2 = np.array([[5, 6], [7, 8]])

# Stacking vertically (along rows)
stacked_v = np.vstack((arr1, arr2))

print("Vertical Stacking:")
print(stacked_v)

# Stacking horizontally (along columns)
stacked_h = np.hstack((arr1, arr2))

print("Horizontal Stacking:")
print(stacked_h)

Differences Between Stacking and Concatenation:
While stacking and concatenation may appear similar, they have key differences. Here are 3 differences between them

Axis of Operation:

Stacking: Stacks along a new axis (e.g., np.vstack() adds rows, np.hstack() adds columns, np.dstack() adds depth).
Concatenation: Combines along an existing axis (e.g., np.concatenate() merges arrays along specified axis).
New Dimension vs. Existing Dimension:

Stacking: Adds a new dimension to the arrays. For example, stacking creates a higher-dimensional array.
Concatenation: Merges arrays along an existing axis without adding a new dimension.
Shape Compatibility:

Stacking: Can stack arrays of different shapes as long as the dimensions are compatible for the stacking operation
Concatenation: Requires that the arrays have the same shape along all axes except for the one being concatenated.
```